

Assignment 1 – Text Processing Fundamentals using MapReduce

Lucija Aleksic, Filip Aleksic, Anabel Dautovic, Ana Petrovic, Maty Prechl

April 29, 2025

1 Introduction

This assignment is used to familiarize us with the MapReduce programming model through processing a huge collection of Amazon reviews. The target of the exercise is to find most used terms in a review per category. That will give us further insight into each of the categories. The main challenge of the exercise is to perform all processing by using a MapReduce framework which we will run on the Hadoop cluster at TU Wien.

We will use the `mrjob` package from python which we will use to create a series of jobs to be ran with a shell script on the cluster. First job will be to run preprocessing, after that we need to count the tokens and categories. Then we can run the job to perform the chi-squared test. Finally we can run the job to find top N terms and an another job to generate and format the output.

2 Preprocessing and execution

As mentioned, the first job is concerned with preprocessing the reviews. There is an example of the JSON of one of the reviews below.

```
{
  "reviewerID": "A2VNYWOPJ13AFP",
  "asin": "0981850006",
  "reviewerName": "Amazon Customer \"carringt0n\"",
  "helpful": [6, 7],
  "reviewText": "This was a gift for my other husband. He's making us things from it all the
    time and we love the food. Directions are simple, easy to read and interpret, and fun
    to make. We all love different kinds of cuisine and Raichlen provides recipes from
    everywhere along the barbecue trail as he calls it. Get it and just open a page. Have
    at it. You'll love the food and it has provided us with an insight into the culture
    that produced it. It's all about broadening horizons. Yum!!",
  "overall": 5.0,
  "summary": "Delish",
  "unixReviewTime": 1259798400,
  "reviewTime": "12 3, 2009",
  "category": "Patio_Lawn_and_Garde"
}
```

As said in the assignment description, we will use the *Amazon Review Dataset 2014*, which contains 142.8 million reviews from 24 product categories.

To be able to use them we need to clean and prepare the raw Amazon data. The `preprocess.py` script handles this task using a MapReduce approach:

- *Input*: JSON lines of Amazon product reviews
- *Process*:

1. Each review is parsed and its text is converted to lowercase
 2. Text is tokenized by splitting on various delimiters (spaces, punctuation, etc.)
 3. Stopwords (common words with little discriminative value) are removed using a provided stopwords file
 4. Single-character tokens are filtered out to reduce noise
- *Output:* JSON lines containing the review category and filtered review text

The mapper loads the stopwords into memory for efficient filtering and processes each review independently. This allows the task to be distributed across multiple nodes in the Hadoop cluster, enabling efficient processing of large datasets.

3 Counting tokens and categories

After the preprocessing step we are left with JSON files with cleaned data. As the next step is to calculate various statistics we need to aggregate documents per category and token to use. This will help us with calculating the Chi-Square values.

The `token_category_counts.py` script does as follows:

- *Input:* Preprocessed review data
- *Process:* For each review, the mapper emits three types of counts:
 1. Documents per category: `(\("__CATEGORY_DOC__", category), 1)`
 2. Documents per token: `( "__TOKEN\__DOC__", token), 1)`
 3. Documents per (category, token) pair: `((category, token), 1)`
- *Output:* Aggregated counts for each key type

The combiner and reducer sum these counts, producing global statistics about:

1. How many documents belong to each category
2. How many documents contain each specific token
3. How many documents contain a specific token within a specific category

These counts form the basis for the contingency tables used in the Chi-Square calculation.

4 Calculating Chi-Square values

By using the previous steps' counts we can proceed with calculating the Chi-Squared test values for each (category, token) pair. The Chi-Squared test is used because it calculates observed frequencies against the expected frequencies. In other words, it tells us how characteristic and important a word is for a given category.

The `chi_square.py` script looks like this:

- *Input:* The counts generated in previous step
- *Process:* For each (category, token) pair, calculate the Chi-Square value using:
 1. A: Documents where token appears in this category
 2. B: Documents where token appears, but not in this category
 3. C: Documents in this category where token does NOT appear
 4. D: Documents where token does NOT appear and are NOT in this category

The formula used is: $X^2 = N((AD - BC)^2)/((A + B)(C + D)(A + C)(B + D))$, where N is the total number of documents

- *Output:* Chi-Square values for each (category, token) pair

The results of this step will give us a value per word which tells us how strongly it is associated with the category. The higher the Chi-Squared value is, the more characteristic the word is for that category. For example, for the category *Pet Supplies* we can expect the word *cat* to have a higher Chi-Squared value than *cable*.

5 Selecting top \$TOP_N terms per category

Now that we have the relevant measure for each (category, token) pairs we can find the most discriminative terms for each category.

The `top75_per_category.py` script handles this task in the following way:

- **Input:** Chi-Square values
- *Process:*
 1. The mapper reorganizes the data so that tokens and scores are grouped by category
 2. The reducer sorts tokens by their Chi-Square score for each category
 3. Only the top N tokens with the highest scores are kept (default is 75)
- *Output:* For each category, the top N tokens with their Chi-Square scores

In this step the MapReducer framework helps us significantly reduce the data volume by grouping categories in the mapper and keeping only the TOP_N terms in the reducer. Now that all the processing work is done, we need to make the outputs human-readable, which will be the focus of the next step.

6 Generating final output

The `generate_output.py` script which will generate the output looks like so:

- *Input:* Top N terms per category
- *Process:*
 1. Collects all tokens for each category and sorts them by score
 2. Formats the output as required: "**category token1 token2 ...**"
 3. Also creates a merged dictionary of all unique tokens (excluding stopwords)
- *Output:* A formatted list of categories with their top discriminative terms and scores, plus a merged dictionary

This final output provides valuable insights into which terms are most characteristic for each product category. This is an example from our output:

```
"\"Apps_for_Android\" \"games:2537037.1056 play:2247115.1857 graphics:1713265 ..."
```

This tells us that in the *Apps for Android* category the three most important words are *games*, *play* and *graphics* with the Chi-Squared scores 2537037.1056, 2247115.1857 and 1713265 respectively. This gives us insight in what users are most interested and affected by. In this example, the users are more interested in games for Android than some other kinds of app. That can help us better target the market and users, if desired.

7 Pipeline

The entire pipeline is orchestrated by a shell script `run_pipeline.sh`, which:

1. Cleans up any previous outputs
2. Executes each MapReduce job in sequence
3. Manages the Hadoop streaming commands and file paths
4. Allows customization of parameters like the number of top terms

This modular design makes the pipeline flexible for future modifications if we would want to add or remove some job, for example add a job to find users who are most engaging.

To execute the whole task from start to finish and run the pipeline we first need to transfer the files and log in. That is done by using running:

```
% scp assignment1.zip {$USER}@lbd.tuwien.ac.at:~
% ssh {$USER}@lbd.tuwien.ac.at
$ unzip assignment1.zip
```

When the files are on the cluster, and we're on the cluster as well, we can create the necessary files on the Hadoop Distributed File System. That is done by using:

```
$ hdfs dfs -mkdir -p input
$ hdfs dfs -put data/stopwords.txt input/stopwords.txt
```

Now we can access the input data and the stopwords file in the pipeline script for them to be used.

To actually run the script, we first need to allow it and the python scripts to be executed, and then run it with the necessary input parameters.

```
$ chmod +x src/run_pipeline.sh
$ chmod +x src/*.py
$ run_pipeline.sh \
  hdfs:///user/dic25_shared/amazon-reviews/full/reviewscombined.json \
  hdfs:///user/{$USER}/input/stopwords.txt \
  output_full \
  75
```

As defined in the script, the first parameter is the input path of the data, the second is the stopwords, third defines the output and the last one is the TOP_N value which selects how many terms are used per category.

After the scripts' execution we need to retrieve the output. For that we need to get the file from HDFS to the cluster file system, and then copy it locally.

```
$ hdfs dfs -get output_full/part-00000 ~/output_full.txt
$ scp {$USER}@lbd.tuwien.ac.at:~/output_full.txt .
```

The time it took for our script to run was about 30 minutes. That can be due to the actual imperfections in our approach, but also to the clusters' current load.